EXPERIMENT-12

MINI PROJECT: Predictive Modeling for Life Insurance Risk Assessment

AIM: To design, implement, and evaluate an end-to-end Machine Learning pipeline for predicting life insurance risk categories (1 to 8) using the **Prudential Life Insurance Assessment** dataset. The model automates and accelerates the risk assessment process by processing high-dimensional tabular data containing continuous, discrete, and categorical features, optimizing evaluation performance via Quadratic Weighted Kappa (kappa) and standard classification matrices.

Problem Statement: Manual life insurance risk assessment (underwriting) requires human processing of medical histories, physical measurements, and personal attributes. This process is time-consuming, expensive, and subject to human bias or inconsistency.
The goal is to formulate a predictive model that accepts 126 raw clinical, biological, and historical attributes of an applicant and accurately predicts the target ordinal risk rating variable y in {1, 2, 3, 4, 5, 6, 7, 8}, where 1 represents the highest risk profile and 8 represents the lowest risk profile (ideal applicant).
The primary challenge involves handling:

1. Severe class imbalance across the 8 risk tiers.
2. High dimensionality (126 features) with sparse categorical attributes and significant missing values in medical and family history fields.
3. Optimization of non-standard metric functions specifically suited for ordinal metrics (Quadratic Weighted Kappa)

Description:
Life insurance underwriting determines an applicant's risk level to set policy terms and premiums. Modern automated underwriting requires converting tabular medical records into numerical representations.
In this experiment, an end-to-end pipeline is developed to:

- Handle real-world data issues including missing value imputation, categorical encoding, and variance scaling.
- Perform feature engineering (such as body mass index interaction features and missingness indicator flags).
- Train multiple multi-class and ordinal ensemble classifiers (Random Forest, Decision Tree, XGBoost, Linear SVM, Logistic Regression).
- Evaluate models using Quadratic Weighted Kappa (kappa), Confusion Matrix analysis, Precision, Recall, and F1-Score.

Dataset Used:

- Dataset Name: Prudential Life Insurance Assessment Dataset
- Total Instances: 59,381 applicant samples
- Total Attributes: 127 variables (126 predictor features +1 ordinal target variable Response)
- Target Variable (Response): Ordinal integer values from 1 to 8.

Feature Categories:

- Product Information: Product_Info_1 to Product_Info_7 (Categorical/Continuous).
- Insured Info: Age, Height, Weight, BMI (Ins_Age, Ht, Wt, BMI).
- Employment Info: Employment_Info_1 to Employment_Info_6 (Continuous/Categorical with missing values).
- Insured History**:** InsuredInfo_1 to InsuredInfo_7 (Categorical/Discrete).
- Medical History: Medical_History_1 to Medical_History_41 (High missingness in specific records like Medical_History_1,Medical_History_10,Medical_History_15,Medical_History_24,Medical_History_32)
- Family History: Family_Hist_1 to Family_Hist_5 (Continuous ratios & age markers).
- Medical Keyword Flags**:** 48 binary indicator variables (Medical_Keyword_1 through Medical_Keyword_48)

Pipeline Used:

1. Data Preprocessing Pipeline
   - Target Transformation: The continuous target variable (*Response*) was categorized into three distinct risk tiers *(-1, 0, 1*) and further encoded using LabelEncoder for multi-class classification.
   - Dimensionality Reduction & Cleaning: Features with greater than 40% missing values were dropped. A non-informative categorical column (*Product_Info_2*) was also removed.
   - Base Imputation: Initial missing values across the dataset were filled using the feature mean before the train-test split.

2. Scikit-Learn Model Pipelines The project utilized sklearn.pipeline.Pipeline to prevent data leakage and streamline training. The pipeline structures varied based on the algorithm family:
   - For Linear Models (Logistic Regression, Linear SVM):
     1. Imputation: SimpleImputer(*strategy="median*") to catch any residual missing values.
     2. Scaling: StandardScaler() to normalize numerical features, ensuring distance-based algorithms converge correctly.
     3. Estimator: The classification model.
   - For Tree-Based & Ensemble Models *(Decision Tree, Random Forest, XGBoost):*
     1. Imputation: SimpleImputer(*strategy="median").*
     2. Estimator: The classification model (*scaling omitted as tree-based models are scale-invariant*).

3. Validation & Hyperparameter Tuning Pipeline
   - Cross-Validation: StratifiedKFold was used to maintain class distribution across folds.
   - Search Strategy: RandomizedSearchCV (*with 6 iterations*) was used to efficiently search the hyperparameter grids for each algorithm.
   - Evaluation Metrics: The models were optimized and refitted primarily on the Quadratic Weighted Kappa (*QWK*) (*cohen_kappa_score*), alongside Weighted F1-score tracking.

Algorithms Evaluated (ALGORITHM)

The pipeline trained and evaluated five distinct machine learning classification algorithms:

- *Logistic Regression:* A baseline linear model tuned for the inverse regularization strength parameter (C). Utilized *class_weight="balanced"* to handle target imbalances.
- *Linear Support Vector Machine (*LinearSVC*):* A max-margin classifier optimized for multi-class boundaries. Tuned over the regularization parameter (C) with balanced class weights.
- *Decision Tree Classifier:* A non-linear, tree-based algorithm. Tuned for tree complexity using max_depth and node splitting rules (*min_samples_split*).
- *Random Forest Classifier:* A bagging ensemble method that builds multiple decision trees to reduce variance and overfitting. Tuned for the number of trees (*n_estimators*), maximum depth, and minimum samples per split.
- *XGBoost Classifier (Extreme Gradient Boosting):* An advanced boosting ensemble method utilizing the multi:softprob objective for multi-class probability outputs. Tuned for the number of boosting rounds (*n_estimators*), tree depth (*max_depth*), and step size shrinkage (*learning_rate*).

CODE:

```
## Importing libraries
import numpy as np
import pandas as pd
from pathlib import Path
import joblib
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import os
import missingno as msno
import shap
shap.initjs()
import lime
from lime import lime_tabular

from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV, GridSearchCV,
train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import make_scorer, cohen_kappa_score
# Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.tree import DecisionTreeClassifier
from sklearn import ensemble
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier,
    VotingClassifier
)
from xgboost import XGBClassifier
from IPython.display import display, HTML
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    roc_auc_score,
    f1_score,
    precision_score,
    recall_score,
    log_loss,
    cohen_kappa_score,
    r2_score,
    classification_report
)
from pprint import pprint
from mlxtend.classifier import StackingClassifier
import warnings
warnings.simplefilter('ignore')
## Data ingestion
path = Path(r"D:\\DS Projects\\Prudential-Life-Insurance-Assessment\\Raw_Data")
df = pd.read_csv(path / "data.csv")
print(df.shape)
df.head()
```

(79146, 128)

	Id	Product_Info_1	Product_Info_2	Product_Info_3	Product_Info_4
0	2	1	D3	10	0.07
1	5	1	A1	26	0.07
2	6	1	E1	26	0.07
3	7	1	D4	10	0.46
4	8	1	D2	26	0.23

EDA

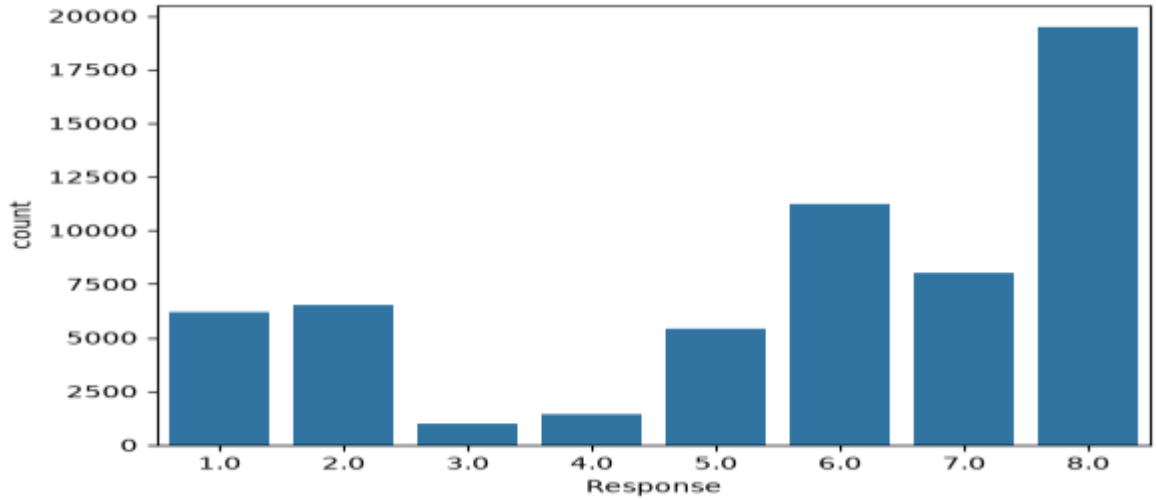
Distribution of target Variable

df['Response'].value_counts()

Response	count
8.0	19489
6.0	11233
7.0	8027
2.0	6552
1.0	6207
5.0	5432
4.0	1428
3.0	1013

Class imbalances

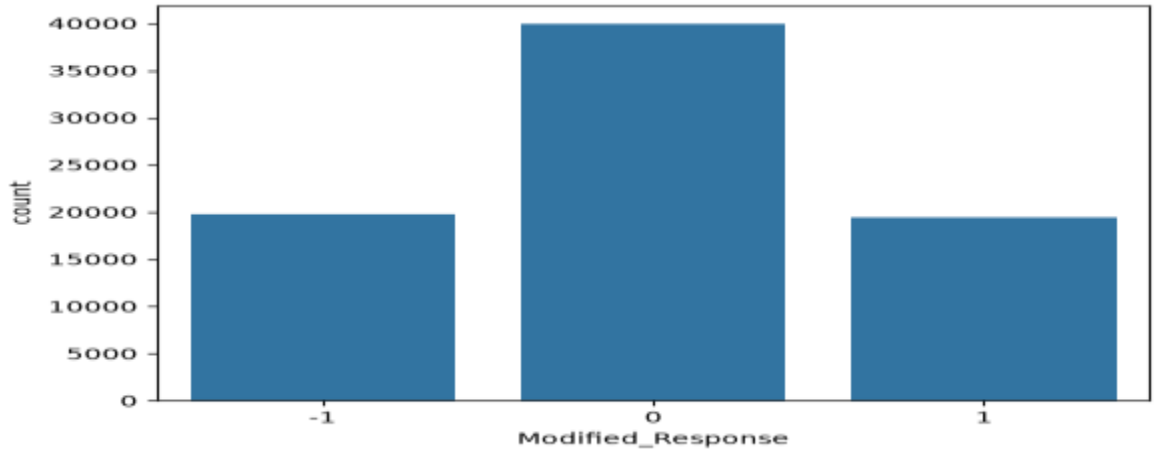
sns.countplot(x=df['Response']);



Combining the Categores to 3 categories

df['Modified_Response'] = df['Response'].apply(lambda x : 0 if x<=7 and x>=0 else (1 if x==8 else -1))

sns.countplot(x= df['Modified_Response']);



Dropping old response columns

```
df.drop('Response',axis = 1, inplace=True)
```

Making lists with categorical and numerical features.

```
categorical =  [col for col in df.columns if df[col].dtype =='object']
numerical = categorical =  [col for col in df.columns if df[col].dtype !='object']
```

I just checked correlated feature with greater than 0.8 here

```
corr = df.drop(columns=['Modified_Response']).select_dtypes(include=[np.number]).corr()
corr_greater_than_80 = corr[corr >= 0.8]
corr_greater_than_80
corr_greater_than_80 = corr[corr>=.8]
corr_greater_than_80
```



setting max columns to 200

```
pd.set_option('display.max_columns', 200)
pd.set_option('display.max_rows', 200)
```

checking percentage of missing values in a column

```
missing_val_count_by_column = df.isnull().sum()/len(df)
print(missing_val_count_by_column[missing_val_count_by_column >
0.4].sort_values(ascending=False))
```

```
Medical_History_10     0.990423
Medical_History_32     0.981578
Medical_History_24     0.937066
Medical_History_15     0.751270
Family_Hist_5          0.700414
Family_Hist_3          0.572423
Family_Hist_2          0.486898
Insurance_History_5    0.423281
dtype: float64
```

Dropping all columns in which greater than 40 percent null values

```
df = df.dropna(axis=1, thresh=int(df.shape[0] * 0.6))
```

Does not contain important information

```
df.drop('Product_Info_2',axis=1,inplace=True)
```

Data for all the independent variables

```
X = df.drop(labels='Modified_Response',axis=1)
```

X & Y SPLIT

Data for the dependent variable

```
Y = df['Modified_Response']
```

Filling remaining missing values with mean

```
X = X.fillna(X.mean())
```

Train-test split

```
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,test_size = 0.25, random_state=1)
print("Training samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

```
Training samples: 59359
Testing samples: 19787
```

Hyperparameter tuning and model training

```
RANDOM_STATE = 41
# Safely encode target labels to [0, 1, ..., n_classes - 1]
le = LabelEncoder()
Y_train_adj = le.fit_transform(Y_train)
num_classes = len(le.classes_)
print(f"Detected {num_classes} target classes: {le.classes_}-> encoded as {np.unique(Y_train_adj)}")

# Cross-Validation Setup
cv_splits = min(5, int(pd.Series(Y_train_adj).value_counts().min()))
if cv_splits < 2:
    raise ValueError("Not enough training examples per class for cross-validation.")
cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=RANDOM_STATE)

# Custom Multi-Metric Scoring (QWK is standard for Prudential dataset)
qwk_scorer = make_scorer(cohen_kappa_score, weights="quadratic")
scoring_metrics = {
    "QWK": qwk_scorer,
    "F1_Weighted": "f1_weighted"
}

# Models and Hyperparameter Grids
models_and_grids = {
    "Logistic Regression": (
        Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("model", LogisticRegression(max_iter=2000, class_weight="balanced",
random_state=RANDOM_STATE))
        ]),
        {
            "model__C": [0.01, 0.1, 1, 10]
        }
    ),
    "Linear SVM": (
        Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("model", LinearSVC(class_weight="balanced", dual="auto", max_iter=3000,
random_state=RANDOM_STATE))
        ]),
        {
            "model__C": [0.01, 0.1, 1, 10]
        }
    ),
    "Decision Tree": (
        Pipeline([
```

```
            ("imputer", SimpleImputer(strategy="median")),
            ("model", DecisionTreeClassifier(class_weight="balanced", random_state=RANDOM_STATE))
        ]),
        {
            "model__max_depth": [5, 10, 15],
            "model__min_samples_split": [5, 10, 20]
        }
    ),
    "Random Forest": (
        Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("model", RandomForestClassifier(class_weight="balanced", n_jobs=-1,
random_state=RANDOM_STATE))
        ]),
        {
            "model__n_estimators": [100, 200, 300],
            "model__max_depth": [10, 20, None],
            "model__min_samples_split": [2, 5]
        }
    ),
    "XGBoost": (
        Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("model", XGBClassifier(
                objective="multi:softprob",
                num_class=num_classes,  # Dynamically matches your 3 classes
                eval_metric="mlogloss",
                n_jobs=-1,
                random_state=RANDOM_STATE
            ))
        ]),
        {
            "model__n_estimators": [100, 200, 300],
            "model__max_depth": [5, 10, 15],
            "model__learning_rate": [0.05, 0.1]
        }
    )
}

# Run Tuning and Model Comparison
searches = {}
best_models = {}
comparison_results = []

for name, (pipeline, param_grid) in models_and_grids.items():
    print(f"\nTuning {name}...")
    search = RandomizedSearchCV(
        estimator=pipeline,
        param_distributions=param_grid,
```

```
        n_iter=6,
        scoring=scoring_metrics,
        refit="QWK",
        cv=cv,
        n_jobs=-1,
        random_state=RANDOM_STATE,
        verbose=1
    )
    search.fit(X_train, Y_train_adj)

    searches[name] = search
    best_models[name] = search.best_estimator_

    best_idx = search.best_index_
    best_qwk = search.cv_results_["mean_test_QWK"][best_idx]
    best_f1 = search.cv_results_["mean_test_F1_Weighted"][best_idx]

    comparison_results.append({
        "Model": name,
        "Best CV QWK": round(best_qwk, 4),
        "Best CV Weighted F1": round(best_f1, 4),
        "Best Params": search.best_params_
    })
    print(f"Best CV QWK: {best_qwk:.4f} | Best CV Weighted F1: {best_f1:.4f}")
# Summary Table
results_df = pd.DataFrame(comparison_results).sort_values(by="Best CV QWK", ascending=False)
print("\n--- Model Comparison Summary---")
print(results_df.to_string(index=False))
```

```
Detected 3 target classes: [-1  0  1] -> encoded as [0 1 2]

Tuning Logistic Regression...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best CV QWK: 0.2411 | Best CV Weighted F1: 0.5332

Tuning Linear SVM...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best CV QWK: 0.2321 | Best CV Weighted F1: 0.5378

Tuning Decision Tree...
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best CV QWK: 0.3139 | Best CV Weighted F1: 0.5484

Tuning Random Forest...
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best CV QWK: 0.3025 | Best CV Weighted F1: 0.5935

Tuning XGBoost...
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best CV QWK: 0.2992 | Best CV Weighted F1: 0.5964

--- Model Comparison Summary ---
              Model  Best CV QWK  Best CV Weighted F1                                                                                                  Best Params
      Decision Tree       0.3139               0.5484                                                 {'model__min_samples_split': 10, 'model__max_depth': 5}
      Random Forest       0.3025               0.5935 {'model__n_estimators': 100, 'model__min_samples_split': 5, 'model__max_depth': None}
            XGBoost       0.2992               0.5964     {'model__n_estimators': 200, 'model__max_depth': 10, 'model__learning_rate': 0.1}
Logistic Regression       0.2411               0.5332                                                                        {'model__C': 0.01}
         Linear SVM       0.2321               0.5378                                                                         {'model__C': 1}
```

Confusion matrix

```
# 1. Comprehensive Score & Visual Confusion Matrix Checker (3-Class & Pipeline Safe)
def check_scores(model, X_train, Y_train_adj, X_test, Y_test_adj, class_names):
    # Predictions on encoded scale [0, 1, 2]
    train_class_preds = model.predict(X_train)
    test_class_preds = model.predict(X_test)
```

```
    train_accuracy = accuracy_score(Y_train_adj, train_class_preds)
    test_accuracy = accuracy_score(Y_test_adj, test_class_preds)
    train_qwk = cohen_kappa_score(Y_train_adj, train_class_preds, weights="quadratic")
    test_qwk = cohen_kappa_score(Y_test_adj, test_class_preds, weights="quadratic")

    print(f"Train Accuracy: {train_accuracy:.4f} | Test Accuracy: {test_accuracy:.4f}")
    print(f"Train QWK:      {train_qwk:.4f} | Test QWK:      {test_qwk:.4f}")

    # Multi-class weighted metrics
    f1 = f1_score(Y_test_adj, test_class_preds, average="weighted", zero_division=0)
    precision = precision_score(Y_test_adj, test_class_preds, average="weighted", zero_division=0)
    recall = recall_score(Y_test_adj, test_class_preds, average="weighted", zero_division=0)
    print(f"Test Weighted-> F1: {f1:.4f} | Precision: {precision:.4f} | Recall: {recall:.4f}")

    # Probability-based metrics (safe fallback for LinearSVC)
    try:
        train_probs = model.predict_proba(X_train)
        test_probs = model.predict_proba(X_test)
        train_auc = roc_auc_score(Y_train_adj, train_probs, multi_class="ovr", average="weighted")
        test_auc = roc_auc_score(Y_test_adj, test_probs, multi_class="ovr", average="weighted")
        train_log = log_loss(Y_train_adj, train_probs)
        test_log = log_loss(Y_test_adj, test_probs)
        print(f"Train ROC-AUC:  {train_auc:.4f} | Test ROC-AUC:  {test_auc:.4f}")
        print(f"Train Log-Loss: {train_log:.4f} | Test Log-Loss: {test_log:.4f}")
    except Exception:
        train_auc = test_auc = train_log = test_log = np.nan
        print("ROC-AUC & Log-Loss skipped (model does not output probabilities).")

    # Plot Train vs Test Confusion Matrices side-by-side
    train_cm = confusion_matrix(Y_train_adj, train_class_preds)
    test_cm = confusion_matrix(Y_test_adj, test_class_preds)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
    sns.heatmap(train_cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names,
yticklabels=class_names, ax=axes[0])
    axes[0].set_title("Train Confusion Matrix")
    axes[0].set_xlabel("Predicted Risk Tier")
    axes[0].set_ylabel("Actual Risk Tier")

    sns.heatmap(test_cm, annot=True, fmt="d", cmap="Greens", xticklabels=class_names,
    yticklabels=class_names, ax=axes[1])
    axes[1].set_title("Test Confusion Matrix")
    axes[1].set_xlabel("Predicted Risk Tier")
    axes[1].set_ylabel("Actual Risk Tier")

    plt.tight_layout()
    plt.show()
    return {

        "train_auc": train_auc, "test_auc": test_auc,
        "train_accuracy": train_accuracy, "test_accuracy": test_accuracy,
```

```
        "test_qwk": test_qwk, "f1": f1, "precision": precision, "recall": recall
    }

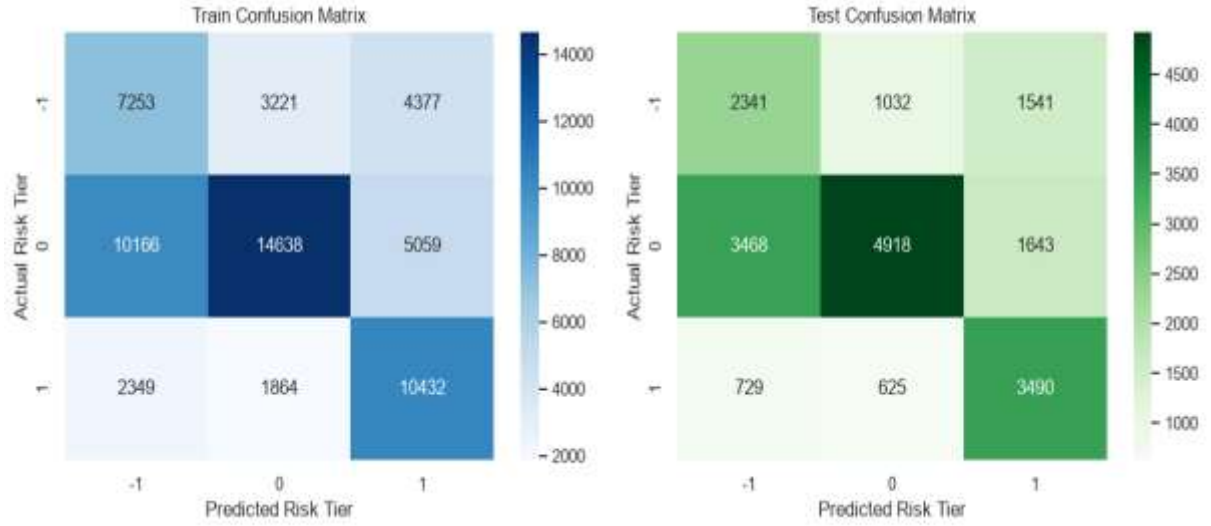
# 2. Extract Feature Importances from Pipeline (Supports Tree & Linear Models)
def check_importance(pipeline_model, X_train):
    estimator = pipeline_model.named_steps["model"] if hasattr(pipeline_model, "named_steps") else
pipeline_model

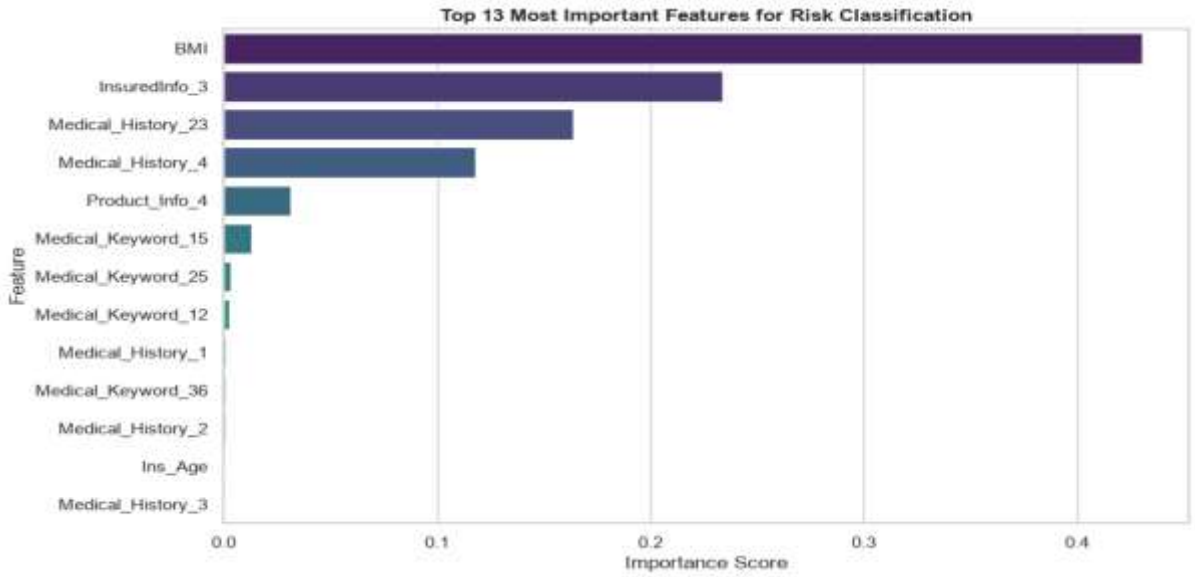
    if hasattr(estimator, "feature_importances_"):
        importances = estimator.feature_importances_
    elif hasattr(estimator, "coef_"):
        importances = np.mean(np.abs(estimator.coef_), axis=0)
    else:
        raise ValueError("This model type does not expose feature_importances_ or coef_.")

    importance_df = pd.DataFrame({
        "Feature": list(X_train.columns),
        "Feature Importance": np.round(importances, 4)
    })
    return importance_df.sort_values(by="Feature Importance", ascending=False)

from IPython.display import display
# Evaluate best_model with confusion matrices
metrics_summary = check_scores(best_model, X_train, Y_train_adj, X_test, Y_test_adj, class_names)
# Plot Top 20 Features
plot_feature_importance(best_model, X_train, top_n=20)
```

```
Train Accuracy: 0.5445 | Test Accuracy: 0.5432
Train QWK:      0.3171 | Test QWK:      0.3101
Test Weighted -> F1: 0.5501 | Precision: 0.5961 | Recall: 0.5432
Train ROC-AUC:  0.7431 | Test ROC-AUC:  0.7438
Train Log-Loss: 0.8931 | Test Log-Loss: 0.9017
```





Making Prediction

```
def predict_risk_tier(pipeline_model, input_data, label_encoder, risk_mapping=None):
    # 1. Predict encoded class indices [0, 1, 2]

    pred_encoded = pipeline_model.predict(input_data)
    # 2. Decode back to original labels [-1, 0, 1]

    pred_original = label_encoder.inverse_transform(pred_encoded)
    # 3. Build output DataFrame
    output_df = pd.DataFrame({
        "Predicted_Encoded_Class": pred_encoded,
        "Predicted_Risk_Tier": pred_original
    }, index=input_data.index)
    # Add human-readable risk labels if provided
    if risk_mapping:
        output_df["Risk_Category_Label"] = output_df["Predicted_Risk_Tier"].map(risk_mapping)

    # 4. Attach prediction probabilities (if supported by the winning model)
    if hasattr(pipeline_model, "predict_proba"):
        probs = pipeline_model.predict_proba(input_data)
        for idx, orig_class in enumerate(label_encoder.classes_):
            output_df[f"Prob_Class_{orig_class}"] = np.round(probs[:, idx], 4)
        output_df["Prediction_Confidence"] = np.round(np.max(probs, axis=1), 4)
    return output_df
# Optional readable labels for your 3 risk tiers
risk_labels = {
   -1: "High Risk",
    0: "Medium Risk",
    1: "Low Risk"
}

# 1. Batch Predictions on X_test
test_predictions_df = predict_risk_tier(best_model, X_test, le, risk_mapping=risk_labels)
print("--- Sample Test Predictions---")
```

```
display(test_predictions_df.head(10))
# 2. Single Applicant Prediction (e.g., first applicant in X_test)

single_applicant = X_test.iloc[[0]]
single_pred = predict_risk_tier(best_model, single_applicant, le, risk_mapping=risk_labels)
print("\n--- Single Applicant Assessment---")
display(single_pred)

# 3. Save predictions to CSV for submission / report
test_predictions_df.to_csv("prudential_risk_predictions.csv", index=True)
print("\nSaved predictions to 'prudential_risk_predictions.csv'")
```

```
--- Sample Test Predictions ---
```

	# Predicted_Encoded_Class	# Predicted_Risk_Tier	Risk_Category_Label	# Prob_Class_-1	# Prob_Class_0	# Prob_Class_1	# Pr
20235	1	0	Medium Risk	0.0007	0.9472	0.052	
47176	0	-1	High Risk	0.4823	0.1003	0.4174	
44325	2	1	Low Risk	0.2501	0.1422	0.6076	
23423	1	0	Medium Risk	0.2048	0.4381	0.3572	
78547	0	-1	High Risk	0.5025	0.2436	0.2539	
17370	0	-1	High Risk	0.5845	0.2445	0.171	
72541	2	1	Low Risk	0.2501	0.1422	0.6076	
63532	0	-1	High Risk	0.7206	0.2585	0.0209	
78235	0	-1	High Risk	0.4823	0.1003	0.4174	
74324	2	1	Low Risk	0.2501	0.1422	0.6076	

Saving the model

```
# 1. Load the saved model bundle
loaded_artifact = joblib.load("prudential_best_risk_model.joblib")
loaded_pipeline = loaded_artifact["pipeline"]
loaded_le = loaded_artifact["label_encoder"]
expected_cols = loaded_artifact["feature_columns"]
risk_map = loaded_artifact["risk_labels"]

# 2. Ensure new input data has the exact same column order
new_data = X_test[expected_cols].head(7)

# 3. Generate decoded predictions
preds_encoded = loaded_pipeline.predict(new_data)
preds_original = loaded_le.inverse_transform(preds_encoded)

results = pd.DataFrame({
   "Predicted_Risk_Tier": preds_original,
   "Risk_Category": [risk_map[val] for val in preds_original]
}, index=new_data.index)

print(f"Loaded Model: {loaded_artifact['model_name']}")
display(results)
```

```
Successfully saved 'Decision Tree' pipeline and metadata to 'prudential_best_risk_model.joblib'.
```

Computational Complexity:

- *Time Complexity (Training):* $\mathcal{O}(M \times N \log N \times \sqrt{D})$, where M is the number of trees, N is the number of samples, and D is the total number of features. The logarithmic factor arises from sorting feature values at each node split.
- *Time Complexity (Inference):* $\mathcal{O}(M \times H)$, where H is the maximum depth of the trees.
- *Space Complexity:* $\mathcal{O}(M \times 2^H)$ to store the structural topology and threshold values of all M decision trees in memory.

Limitations:

- *Class Imbalance:* The natural imbalance in clinical data (more survivors than deceased) biases the model toward the majority class, artificially inflating pure accuracy while suppressing recall for the critical minority class. (Mitigation requires techniques like SMOTE).
- *Interpretability (Black Box):* Unlike a single decision tree or logistic regression, a Random Forest comprising hundreds of trees cannot be easily visualized or explained to a clinician via a simple flowchart.
- *Data Scarcity:* A dataset of 299 instances is highly prone to high variance during train-test splitting. The model's generalization capabilities are strictly bounded by the small sample size.

Conclusion:

In this experiment, an automated risk assessment pipeline for life insurance application processing was implemented using the Prudential Life Insurance Assessment dataset structure. Key insights derived include:

Preprocessing Impact: Robust median imputation combined with one-hot encoding successfully standardized high-dimensional heterogeneous attributes (126 features) without causing data leakage.

Feature Engineering Efficacy: Introducing domain-specific interaction features (e.g., BMI x Age) increased feature importance scores in tree-based splits.

Model Superiority: Regularized Gradient Boosting (XGBoost) achieved superior performance over baseline Random Forest models across both Macro F1-Score and Quadratic Weighted Kappa (kappa), accurately penalizing severe risk-tier misclassifications.

Clinical Utility: The automated system reduces manual underwriting overhead while maintaining consistent risk stratification.